SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – PSEUDOCODE WRITING TASK

Course Code:	CSA0305	**Course Title:**	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	**Assessment:**	Assessment Tool 1 – Pseudocode Writing Task
Weightage:	35% of CIA	**Total Marks:**	100
CO5 Statement:	*Construct MST using shortest path algorithms and traverse the graph to model and analyse realworld problem.*		
Student Name:	K.GNANESWAR KUMAR	**Reg. No.:**	192572105
Section / Batch:	BE-CSE(AI)	**Date:**	31/08/2026

Code of Conduct

I _____K.GNANESWAR KUMAR/192572105_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: _k.gnaneswar kumar_____

Instructions

- This test contains 5 Pseudocode Writing Task questions. Total: 100 Marks.
- When a student answer using pseudocode writing task should evaluate based on the ability to design clear, logical, and efficient pseudocode solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Minimum Spanning Tree	Kruskal's Algorithm	1	20
Graph Sorting	Topological Sort	2	20

Shortest Path Algorithm	MST & Dijkstra's Algorithm	3	20
Shortest Path Algorithm	Dijkstra's Algorithm	4	20
Minimum Spanning Tree	Prim's or Kruskal's Algorithm	5	20
	GRAND TOTAL:	**100 Marks**	

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or nonfunctional code
Code Quality & Readability	20	Clean, modular, wellcommented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

KATTAMURI VENKATA SRINIVASAGNANESWAR KUMAR
192572105RA

My Assessments

Course: **CSA0305RA**

PSEUDOCODE WRITING TASK OPEN ✓ Completed

CO5_AT1_Pseudocode Writing Task

5/5 answered

Review →

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2) OPEN ✓ Completed

DEBUGGING & OPTIMIZATION TASK

10/10 answered

Review →

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 1) OPEN ✓ Completed

DEBUGGING & OPTIMIZATION TASK

10/10 answered

Review →

PSEUDOCODE WRITING TASK

← Back

QUESTIONS

- **Q1** Minimum Spanning Tree - Kruskal's Algorithm — 20 marks
- **Q2** Graph - Topological Sort — 20 marks
- **Q3** Shortest Path Algorithm - Minimum Spanning Tree & Dijkstra's — 20 marks
- **Q4** Shortest Path Algorithm - Dijkstra's Algorithm — 20 marks
- **Q5** Minimum Spanning Tree - Prim's or Kruskal's Algorithm — 20 marks

5/5 answered

Q1 Minimum Spanning Tree - Kruskal's Algorithm

20 marks

Write pseudocode for constructing an MST using Kruskal's Algorithm. A city planning department needs to connect multiple zones with underground fiber-optic cables. Each zone is represented as a vertex, and possible cable connections between zones are represented as edges with installation costs (weights).

Constraints:

a. All zones must be connected

b. Total installation cost must be minimized

c. No redundant connections (no cycles) allowed

d. The network must remain fully connected

Your Answer

```
ALGORITHM KruskalMST(Graph G(V, E))
  Input: Graph G with vertices V (zones) and weighted edges E (fiber-optic cable costs)
  Output: Minimum Spanning Tree MST (minimum cost cable network connecting all zones)

  // Step 1: Initialize an empty set for the MST
  MST = empty set

  // Step 2: Initialize Disjoint Set (Union-Find Data Structure)
  FOR each vertex v in V DO
    MAKE-SET(v)
  END FOR

  // Step 3: Sort all edges in non-decreasing order of their weight (installation cost)
  SORT edges E in non-decreasing order by weight

  // Step 4: Process edges to form the tree without cycles
  FOR each edge e = (u, v) in sorted E DO
    // Find the root representative of the sets containing u and v
    IF FIND-SET(u) != FIND-SET(v) THEN
      // If u and v are in different components, add edge to MST
      MST = MST U {e}
      // Union the sets to mark them as connected
      UNION(u, v)
    END IF
  END FOR
```

Save Answer Next →

PSEUDOCODE WRITING TASK

← Back

QUESTIONS

- **Q1** Minimum Spanning Tree - Kruskal's Algorithm — 20 marks
- **Q2** Graph - Topological Sort — 20 marks
- **Q3** Shortest Path Algorithm - Minimum Spanning Tree & Dijkstra's — 20 marks
- **Q4** Shortest Path Algorithm - Dijkstra's Algorithm — 20 marks
- **Q5** Minimum Spanning Tree - Prim's or Kruskal's Algorithm — 20 marks

5/5 answered

Q2 Graph - Topological Sort

20 marks

A DAG represents task dependencies. Write pseudocode for Topological Sort using DFS with cycle detection.

Constraints:

a. Detect cycle before sorting

b. Use DFS approach

Your Answer

```
ALGORITHM TopologicalSortDFS(Graph G(V, E))
   Input: Directed Graph G with vertices V (tasks) and edges E (dependencies)
   Output: Topological order stack of vertices OR message indicating a cycle exists

   // Step 1: Initialize tracking arrays for state visualization
   // UNVISITED = 0, VISITING = 1, VISITED = 2
   FOR each vertex v in V DO
      state[v] = UNVISITED
   END FOR

   Initialize empty Stack topologicalOrder
   hasCycle = FALSE

   // Step 2: Traverse each unvisited node in the graph
   FOR each vertex v in V DO
      IF state[v] == UNVISITED THEN
         DFS(v, state, topologicalOrder, G, hasCycle)
         IF hasCycle == TRUE THEN
            PRINT "Error: Graph contains a cycle (Not a DAG). Topological Sort impossible."
            RETURN NULL
         END IF
      END IF
   END FOR

   RETURN topologicalOrder
```

Save Answer | Next →

PSEUDOCODE WRITING TASK

← Back

QUESTIONS

- Q1 — Minimum Spanning Tree - Kruskal's Algorithm — 20 marks
- Q2 — Graph - Topological Sort — 20 marks
- Q3 — Shortest Path Algorithm - Minimum Spanning Tree & Dijkstra's — 20 marks
- Q4 — Shortest Path Algorithm - Dijkstra's Algorithm — 20 marks
- Q5 — Minimum Spanning Tree - Prim's or Kruskal's Algorithm — 20 marks

5/5 answered

Q3 Shortest Path Algorithm - Minimum Spanning Tree & Dijkstra's

20 marks

Write pseudocode integrating MST + Dijkstra for Smart transportation system.

Constraints:

a. Use MST for infrastructure

b. Use shortest path for routing

Your Answer

```
ALGORITHM SmartTransportationSystem(Graph G(V, E), sourceNode, targetNode)
  Input: Graph G with vertices V (junctions) and weighted edges E (road network construction costs/lengths)
      sourceNode (start location), targetNode (destination)
  Output: Infrastructure MST network and optimal shortest path route from sourceNode to targetNode

  // ------------------------------------------------------------
  // MODULE 1: Infrastructure Network Construction using Prim's MST
  // ------------------------------------------------------------
  FUNCTION BuildInfrastructureNetwork(G)
    Initialize priority queue PQ
    Initialize key array key[] for all vertices to INF
    Initialize parent array parent[] for all vertices to NULL
    Initialize visited array visited[] for all vertices to FALSE

    // Start from an arbitrary initial node (e.g., node 0)
    key[0] = 0
    PQ.insert(0, 0) // (node, key_value)

    WHILE PQ is not empty DO
      u = PQ.extractMin()
      visited[u] = TRUE

      FOR each neighbor v of u with edge weight w DO
        IF visited[v] == FALSE AND w < key[v] THEN
          key[v] = w
          parent[v] = u
          PQ.decreaseKey(v, w)
        END IF
      END FOR
```

Save Answer Next →

PSEUDOCODE WRITING TASK

← Back

QUESTIONS

- **Q1** Minimum Spanning Tree - Kruskal's Algorithm — 20 marks
- **Q2** Graph - Topological Sort — 20 marks
- **Q3** Shortest Path Algorithm - Minimum Spanning Tree & Dijkstra's — 20 marks
- **Q4** Shortest Path Algorithm - Dijkstra's Algorithm — 20 marks
- **Q5** Minimum Spanning Tree - Prim's or Kruskal's Algorithm — 20 marks

5/5 answered

Q4 Shortest Path Algorithm - Dijkstra's Algorithm — 20 marks

Shortest Path with "Toll-Free" Voucher: You have a voucher that allows you to traverse one edge for free (weight = 0). Write pseudocode to find the shortest path from S to D utilizing this voucher optimally.

Your Answer

```
ALGORITHM FractionalKnapsack(items, capacity)
  Input: Array of items, where each item has (weight, profit)
       capacity (maximum weight capacity of the warehouse truck)
  Output: Maximum profit obtained and fractional selection breakdown

  // Step 1: Compute profit-to-weight ratio for each item
  FOR each item i in items DO
    i.ratio = i.profit / i.weight
    i.fractionTaken = 0.0
  END FOR

  // Step 2: Sort items in descending order of their ratio
  SORT items in descending order based on item.ratio

  currentWeight = 0
  totalProfit = 0.0

  // Step 3: Greedily pick items
  FOR each item i in items DO
    IF currentWeight + i.weight <= capacity THEN
      // Full item fits in the knapsack
      i.fractionTaken = 1.0
      currentWeight = currentWeight + i.weight
      totalProfit = totalProfit + i.profit
    ELSE
      // Item cannot fit completely; take the remaining fractional part
      remainingSpace = capacity - currentWeight
      i.fractionTaken = remainingSpace / i.weight
      totalProfit = totalProfit + (i.profit * i.fractionTaken)
      currentWeight = capacity
      BREAK // Knapsack is full
    END IF
  END FOR
```

PSEUDOCODE WRITING TASK

← Back

QUESTIONS

- **Q1** Minimum Spanning Tree - Kruskal's Algorithm — 20 marks
- **Q2** Graph - Topological Sort — 20 marks
- **Q3** Shortest Path Algorithm - Minimum Spanning Tree & Dijkstra's — 20 marks
- **Q4** Shortest Path Algorithm - Dijkstra's Algorithm — 20 marks
- **Q5** Minimum Spanning Tree - Prim's or Kruskal's Algorithm — 20 marks

5/5 answered

Q5 Minimum Spanning Tree - Prim's or Kruskal's Algorithm

20 marks

Second Best Spanning Tree: Design pseudocode to find the "Second Best" MST.

Hint: Find the MST first, then for every edge in the MST, try removing it and finding the next best edge to replace it.

Your Answer

```
    IF secondBestWeight == INF THEN
        PRINT "No Second Best MST exists (graph lacks alternate spanning trees)."
        RETURN NULL, INF
    END IF

    RETURN SecondBestMST_Edges, secondBestWeight
END ALGORITHM


// Helper Function: Standard Kruskal's MST Algorithm
FUNCTION ComputeKruskalMST(Graph G(V, E))
    Initialize empty set MST
    totalWeight = 0

    // Initialize Disjoint-Set Data Structure
    FOR each vertex v in G.V DO
        MAKE-SET(v)
    END FOR

    // Sort edges by weight in ascending order
    SortedEdges = Sort E in non-decreasing order of weight

    FOR each edge (u, v) with weight w in SortedEdges DO
        IF FIND-SET(u) != FIND-SET(v) THEN
            MST = MST U {(u, v)}
            totalWeight = totalWeight + w
            UNION(u, v)
        END IF
    END FOR

    RETURN MST, totalWeight
END FUNCTION
```